



Operating Systems

Unit 1 part 2

Subject Code: CI42

B.E Sem: IV

Credits : 4



Syllabus

Unit I

Introduction: What operating systems do, Computer System organization, Computer System architecture, Operating System structure, Operating System operations. **Operating system structures:** Operating system services, user operating system Interface, System calls, Types of system calls, Operating system structure, System boot.



Operating-System Services

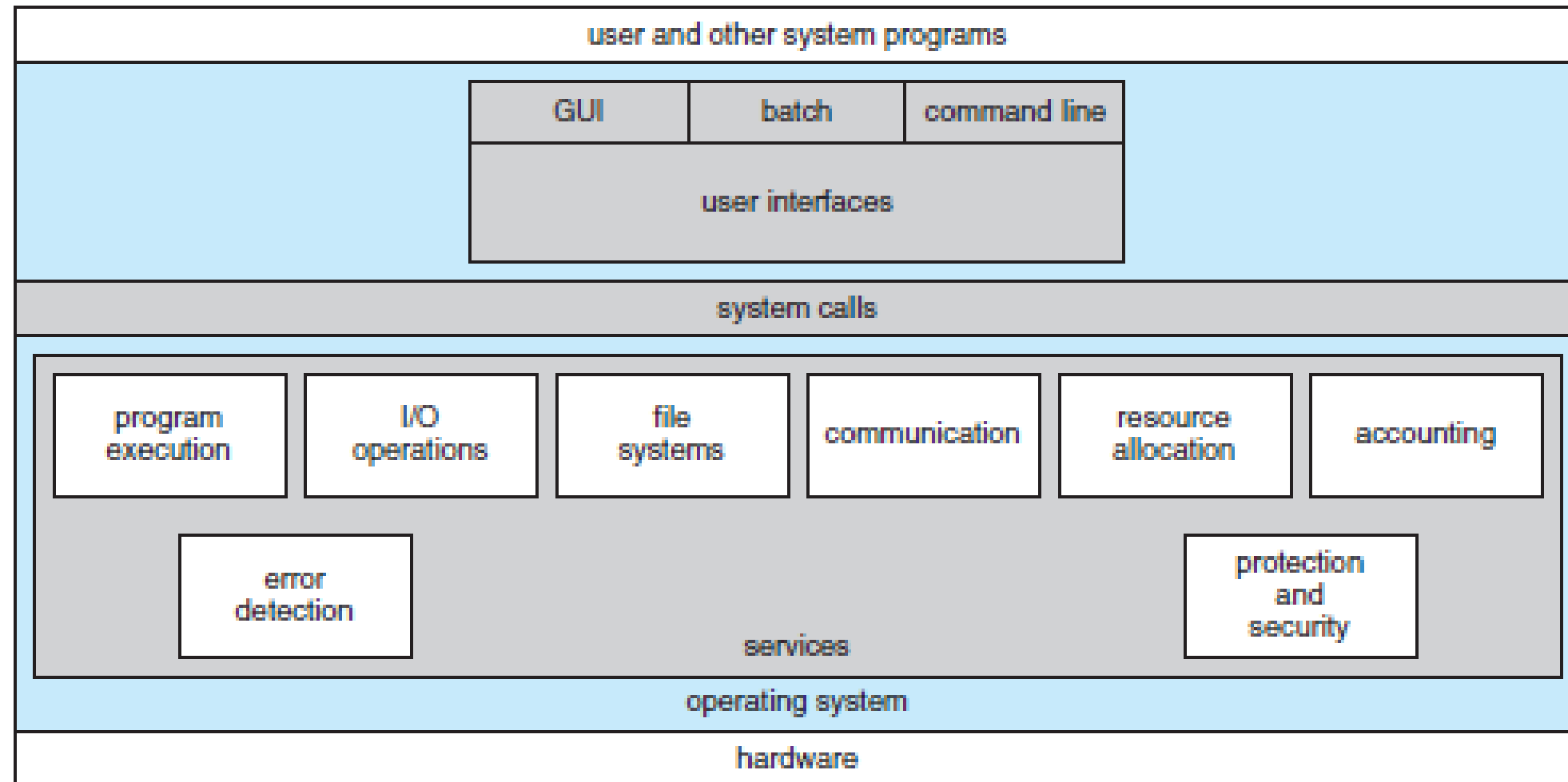


Figure 2.1 A view of operating system services.

The OS provides an environment to execute programs conveniently. It offers specific services to programs and users.

1. User Interface (UI):

- Almost all operating systems have a UI.
- **Command-Line (CLI):** User types text commands (e.g., UNIX, MS-DOS).
- **Batch:** Commands are entered into files and executed automatically.
- **Graphical User Interface (GUI):** Uses windows, icons, menus, and a mouse/touchscreen (e.g., Windows, macOS).



2. Program Execution:

- The system must be able to **load** a program into memory and **run** it.
- It must handle execution termination, whether normal or abnormal (due to an error).

3. I/O Operations:

- A running program may require Input/Output.
- **Protection:** Users cannot control I/O devices directly for efficiency and security; the OS provides the means to do this.



4. File-System Manipulation:

- Programs need to read, write, create, and delete files and directories.
- **Search:** Find files by name.
- **Permissions:** Manage ownership (allow or deny access to specific users).



5. Communications:

- Processes often need to exchange information (on the same computer or across a network).
- **Two Methods:**
 - **Shared Memory:** Processes read and write to a shared section of memory.
 - **Message Passing:** The OS moves packets of information between processes.



6. Error Detection:

- The OS constantly checks for errors to ensure correct computing.
- **Hardware Errors:** Memory error, power failure.
- **I/O Errors:** Parity error on disk, connection failure, printer out of paper.
- **User Program Errors:** Arithmetic overflow, illegal memory access.



These functions exist to ensure the efficient operation of the system itself, especially when sharing resources among multiple users.

1. Resource Allocation:

- When multiple jobs run concurrently, the OS must allocate resources (CPU cycles, main memory, file storage).
- **Scheduling:** The OS decides which program gets the CPU and for how long.



2. Accounting:

- Keeps track of which users use how much and what kinds of computer resources.
- **Usage:** Used for billing users or gathering usage statistics to optimize the system.

3. Protection and Security:

- **Protection:** Controlling access to system resources. One process should not be able to interfere with another or the OS itself.
- **Security:** Authenticating users (passwords) and defending external I/O devices from invalid access attempts.



User and Operating- System Interface



Two Fundamental Interfaces:

- **Command-Line Interface (CLI):** Users directly enter text commands.
- **Graphical User Interface (GUI):** Users interact via windows, menus, and pointers.



Definition:

- The **Command Interpreter** allows users to directly enter commands to be performed by the Operating System.
- On systems with multiple interpreters to choose from, these are known as **Shells**.

Examples:

- **UNIX/Linux:** Bourne shell, C shell, Bourne-Again shell (bash), Korn shell.
- **Windows:** Command Prompt (cmd.exe), PowerShell.



Role:

It acts as the primary interface between the user and the OS kernel on non-GUI systems.

It runs when a job is initiated or when a user logs on.

Primary Function:

To **get** and **execute** the next user-specified command.

Most commands at this level manipulate files (create, delete, list, copy, execute).



How Commands Are Implemented

The Challenge: How does the interpreter know what to do when you type a command (e.g., delete)?

- **Approach 1: Internal Code (Built-in)**

- The command interpreter *contains* the code to execute the command itself.
- **Drawback:** The size of the interpreter is determined by the number of commands.

Adding a command requires reprogramming the interpreter.

- **Approach 2: System Programs (The UNIX Way)**

- The interpreter does **not** understand the command; it treats the command name as a **file name**.

- **Example:** When you type `rm file.txt`:

1. The interpreter searches memory/disk for a file called `rm`.
2. It loads the `rm` file into memory.
3. It executes it using the parameter `file.txt`.

- **Benefit:** Programmers can add new commands easily by simply creating new files. The command interpreter remains small and does not need to be changed.



Brief History:

- **Origin:** Research at **Xerox PARC** (early 1970s). First appeared on the Xerox Alto (1973).
- **Popularization:** The **Apple Macintosh** (1980s) made GUIs widespread.
- **Windows:** Microsoft Windows 1.0 was originally just a GUI running *on top* of MS-DOS.



GUIs on UNIX/Linux:

- Traditionally, UNIX systems were dominated by Command-Line Interfaces (CLI).
- **Commercial GUIs:** Common Desktop Environment (CDE), X-Windows (used on Solaris/AIX).
- **Open Source GUIs:** Linux systems use desktop environments like **KDE** and **GNOME**.
- **Flexibility:** Unlike Windows or macOS, the GUI on Linux is often separate from the kernel, meaning you can choose which GUI you like best.



The Desktop Metaphor:

Instead of typing text commands, users interact with a **window-and-menu** system.

- **Icons:** Images on the screen (the "Desktop") represent programs, files, and directories.
- **Interaction:** Users use a **mouse** to position a pointer. Clicking a button invokes a program or opens a folder.

The Shift to Mobile:

A mouse is impractical for handheld devices (smartphones/tablets).

- **Touchscreen Interface:** Users interact via **gestures** (pressing, swiping fingers).
- **Text Entry:** Physical keyboards are replaced by **virtual keyboards** that appear on the screen when needed.



Preference & Efficiency:

The choice is largely personal preference.

- **System Administrators & Power Users:** Frequently prefer the **Command-Line Interface (CLI)**.
- **Why?** It is often more efficient, providing faster access to deep system functions than clicking through menus.

CLI Advantages:

- **Complete Access:** On some systems, the GUI only exposes a subset of functions. The CLI often allows access to *every* system capability.
- **Automation (Shell Scripts):**
 - CLIs are programmable.
 - **Repetitive Tasks:** A set of command-line steps can be recorded into a file (a shell script) and run just like a program.
 - *Note:* The script is not compiled; it is interpreted by the CLI.



Case Studies:

- **Microsoft Windows:** Most users rely entirely on the GUI environment and rarely use the command shell (cmd.exe or PowerShell).
- **Apple Macintosh:**
 - *Historically:* Mac OS was GUI-only (no command line).
 - *Mac OS X (Modern):* Because it is built on a **UNIX kernel**, it now provides both the **Aqua GUI** and a powerful **Command-Line Interface**.

System Design Philosophy:

- **Kernel View:** The Operating System does not distinguish between "User Programs" (like Chrome) and "System Programs" (like the GUI or CLI). It serves them all equally.
- **Goal:** The OS concentrates on providing services to programs; the UI is just a program that allows humans to request those services.



- It provides an **interface** to the services made available by the Operating System.
- It is the specific mechanism a user program uses to ask the kernel to perform tasks (like accessing hardware, creating a process).

Implementation:

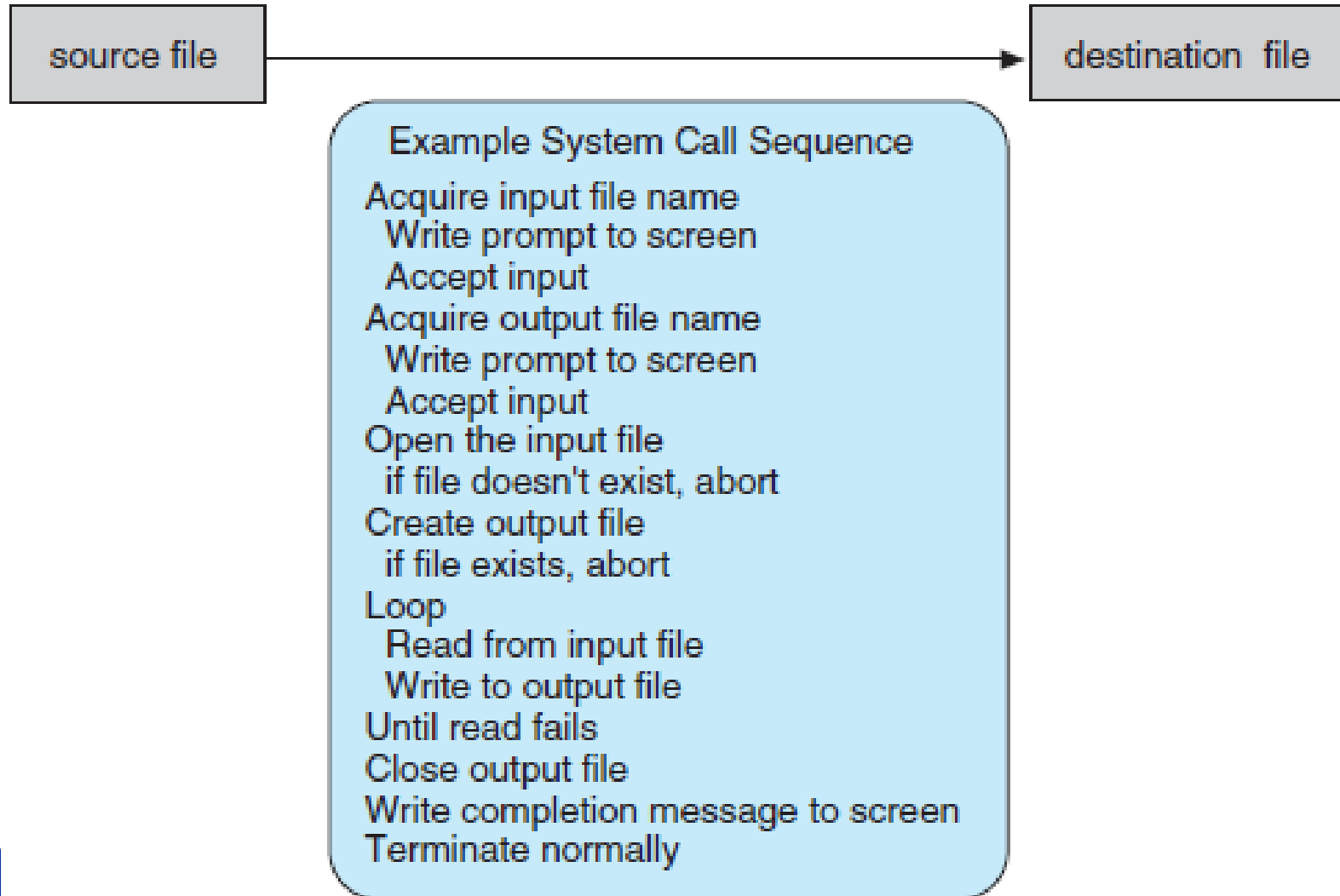
- **High-Level:** Available as routines written in **C** and **C++**.
- **Low-Level:** Certain tasks (e.g., direct hardware access) may require **Assembly-Language** instructions.

The Goal:

- To allow user-level programs to interact with the kernel safely and effectively.

Example of how system calls are used.

The Scenario: Writing a simple program to read data from one file and copy it to another. This requires a long sequence of system calls.





System Calls vs APIs

Feature	Application Programming Interface (API)	System Call
Who uses it?	The Programmer. You write <code>printf()</code> or <code>System.out.println()</code> .	The API Library. It converts your code into a request the OS understands.
Where it lives	User Mode. It runs in the same "safe" space as your application.	Kernel Mode. It runs in the "privileged" space where the OS controls hardware.
Complexity	Simple. Provides easy-to-use functions with clear names.	Complex. Involves low-level CPU registers, interrupts, and memory addresses.
Portability	High. Your code can run on Linux, macOS, or Windows if they share the same API.	Low. System call names and numbers change depending on the specific OS and CPU architecture.
Goal	Convenience. Hides the messy details of the hardware.	Control. Actually performs the task (writing to disk, sending a network packet).



API Execution Example

Programmers access the API via a **System Library** provided by the OS (e.g., libc for C programs).

Example: `printf()` vs. `write()`:

1. **User Code:** The programmer writes `printf("Greetings");`.
2. **Library:** The Standard C Library intercepts this call.
3. **System Call:** The library invokes the `write()` system call in the OS.
4. **Kernel:** The OS executes the write operation and returns the status.
5. **Return:** The library returns the value to the user program.

Key Takeaway: The programmer sees the API (`printf`), but the OS sees the system call (`write`).

The Interface:

- The run-time support system acts as the link between API and the OS.

The Numbering System:

- Each system call has a unique **number**.
- The system-call interface maintains a **table** indexed by these numbers.

The Handshake:

1. The interface places the system call number in a register.
2. It executes a **Trap** (software interrupt) to switch to Kernel Mode.
3. The OS looks up the number in the table and executes the function.
4. Control returns to the user program.

Abstraction: The caller needs to know nothing about the OS internals, just the API function.



The System-Call Interface

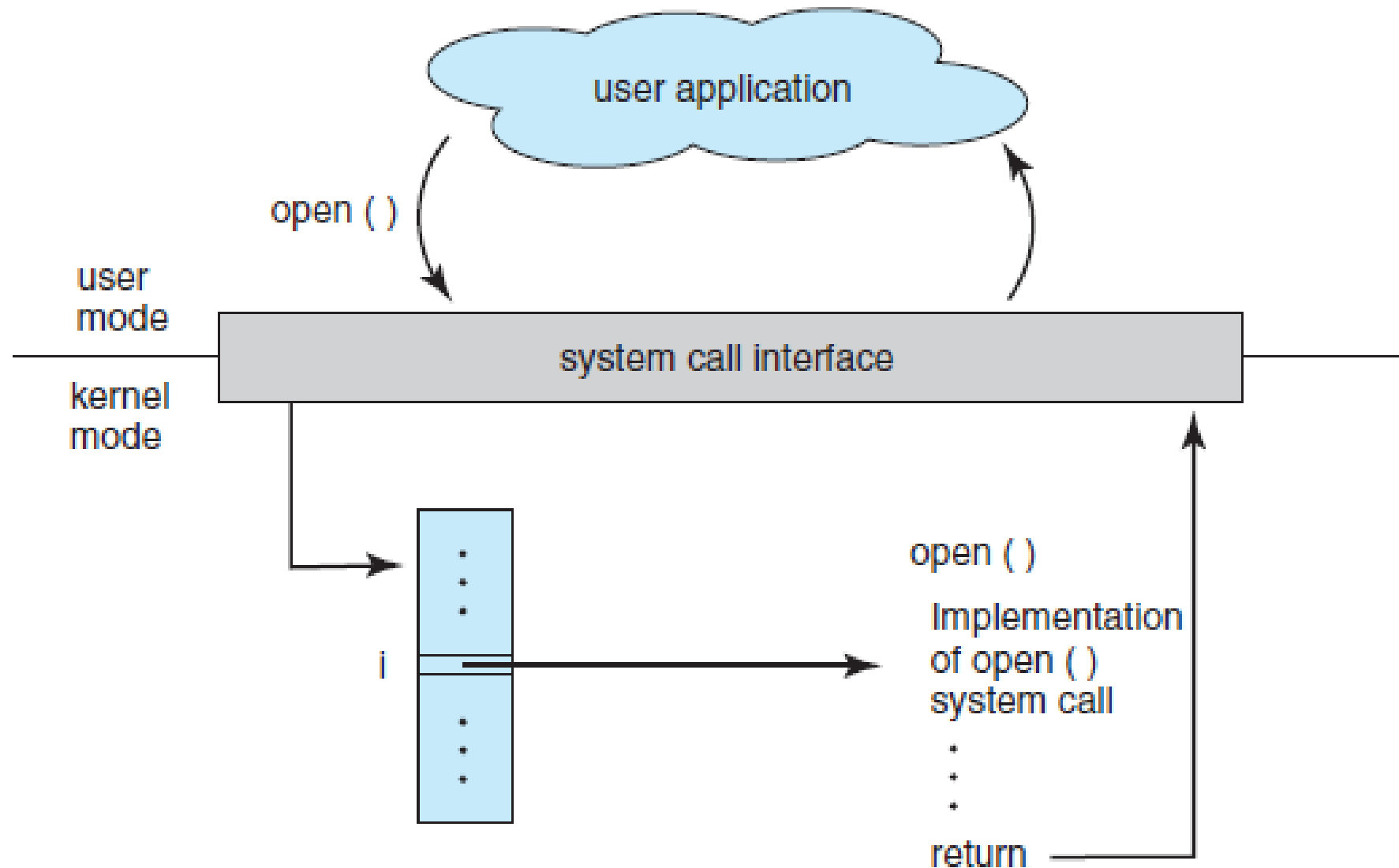


Figure 2.6 The handling of a user application invoking the `open()` system call.



Passing Parameters to the OS

The Challenge: Often, the OS needs more info than just the call number (e.g., *which* file to read, *where* to put the data).

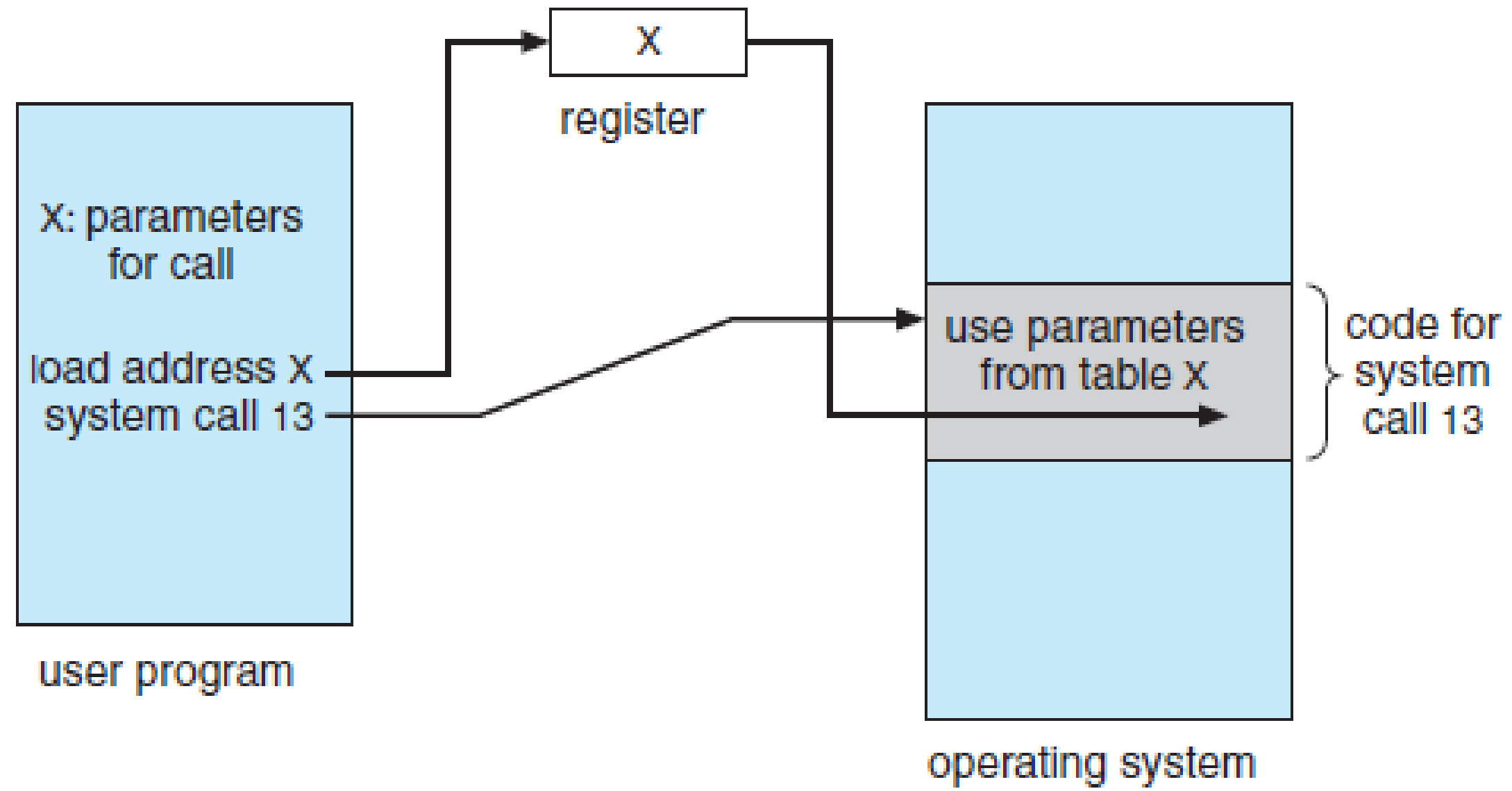
Three Methods:

- 1. Registers:** The simplest/fastest approach. Pass parameters directly in CPU registers. *Limitation: Registers are small.*
- 2. Block/Table:** Parameters are stored in a block in memory. The **address** of the block is passed in a register. (Used by Linux).
- 3. The Stack:** Parameters are **pushed** onto the stack by the program and **popped** off by the OS.

Note: Block and Stack methods are preferred when there is a lot of data to pass.



Passing of parameters as a table





Types of System Calls

System calls are generally grouped into six major categories:

1. **Process Control:** Loading, executing, ending, and waiting for jobs.
2. **File Management:** Creating, reading, and writing files.
3. **Device Management:** Requesting and releasing hardware devices.
4. **Information Maintenance:** Getting time, date, and system data.
5. **Communications:** Sending/receiving messages or shared memory.
6. **Protection:** Controlling access permissions.

Types – Process Control

Functions:

- **End/Abort:** Halting execution normally or due to error.
- **Load/Execute:** A command interpreter starting a new program.
- **Create/Terminate Process:** Spawning new processes (fork in UNIX) or stopping them.
- **Wait/Signal:** Waiting for a specific event or time.
- **Allocate Memory:** Asking the OS for more RAM.

	Windows	Unix
Process Control	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()



Types – File Management

Programs constantly need to interact with file system to save and retrieve data.

These operations apply to both **files** and **directories** (folders).

Common System Calls:

Creation/Deletion:

- **create()**: Requires the name of the file and perhaps initial attributes.
- **delete()**: Removes the file from the system.

Usage:

- **open()**: Prepares a file for use.
- **read() & write()**: Extracts data from or saves data to the file.
- We use **get_file_attributes()** and **set_file_attributes()** to read or change attribute information.



Types – File Management

A process may need access to resources like disk drives, printers, or network interfaces.

- **Access:** request device (to get exclusive use) and release device.
- **Operations:** read, write, reposition (Treating devices similarly to files).
- **Setup:** Logically attach or detach devices.
- **Attributes:** get and set device attributes.

Device
Manipulation

SetConsoleMode()
ReadConsole()
WriteConsole()

ioctl()
read()
write()

Communications (Talking between processes):

- **Connections:** create or delete communication connections.
- **Data Transfer:** send and receive messages (for message-passing models).
- **Remote Hardware:** attach or detach remote devices.
- **Status:** transfer status information.

Protection (System Security):

- **Access Control:** Manipulating permission settings for resources (like files and disks).
- **User Rights:** allow user or deny user access to specific areas of the system.

Communication	CreatePipe()	pipe()
	CreateFileMapping()	shm_open()
	MapViewOfFile()	mmap()
Protection	SetFileSecurity()	chmod()
	InitializeSecurityDescriptor()	umask()
	SetSecurityDescriptorGroup()	chown()



Types – File Management

Programs constantly need to interact with file system to save and retrieve data.

These operations apply to both **files** and **directories** (folders).

Common System Calls:

Creation/Deletion:

- `create()`: Requires the name of the file and perhaps initial attributes.
- `delete()`: Removes the file from the system.

Usage:

- `open()`: Prepares a file for use.
- `read()` & `write()`: Extracts data from or saves data to the file.
- `close()`: Indicates the program is no longer using the file, freeing up resources.

Operating-System Structure

- Simple Structure
- Layered Approach
- Microkernels
- Modules
- Hybrid Systems



Simple Structure

- Many early operating systems did not have well-defined structures. They started small and grew beyond their original scope.
- **MS-DOS** was designed to cram the most functionality into the least amount of space, rather than being carefully divided into modules.

Lack of Separation:

- Interfaces and levels of functionality were not well separated.
- **Direct Access:** Application programs could bypass the OS and write directly to the display and disk drives.

The Vulnerability:

- Because apps had direct access, an errant or malicious program could crash the entire system.
- **Hardware Limitation:** MS-DOS was written for the Intel 8088 chip, which provided no dual-mode operation and no hardware protection.

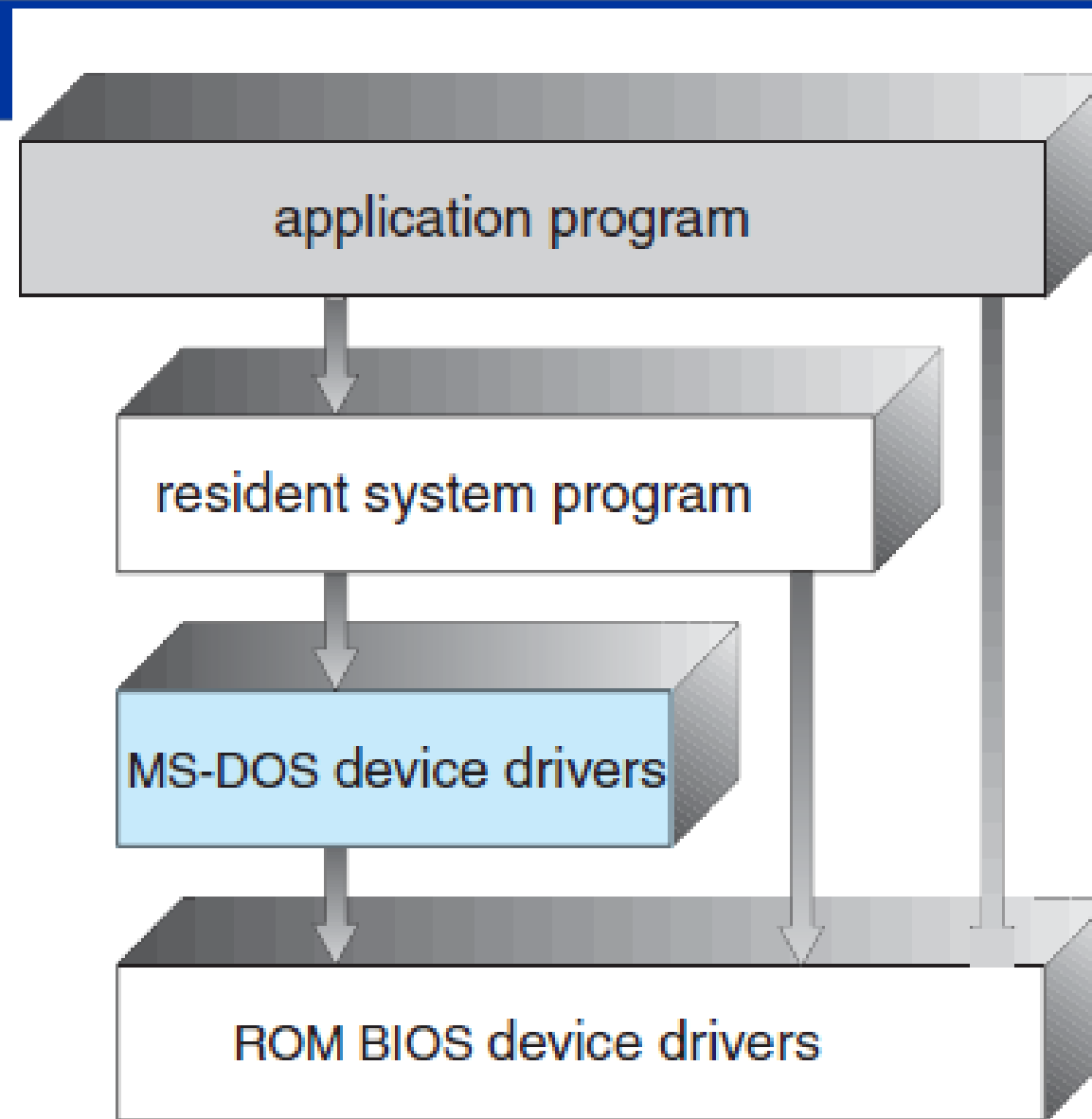


Figure 2.11 MS-DOS layer structure.



Monolithic Structure

- Like MS-DOS, original UNIX was limited by the hardware of its era.
- It separated the OS into two distinct parts: **System Programs** and **The Kernel**.

The Monolithic Kernel:

- The kernel consists of *everything* below the system-call interface and above the physical hardware.
- It combined an enormous amount of functionality into one single level (CPU scheduling, memory management, file systems, device drivers).

The Trade-Off:

- **Disadvantage:** Because it was so massive and interconnected, this monolithic structure was very difficult to implement and maintain.
- **Advantage (Performance):** It had a distinct performance advantage. Because everything is in one place, there is very little communication overhead within the kernel.

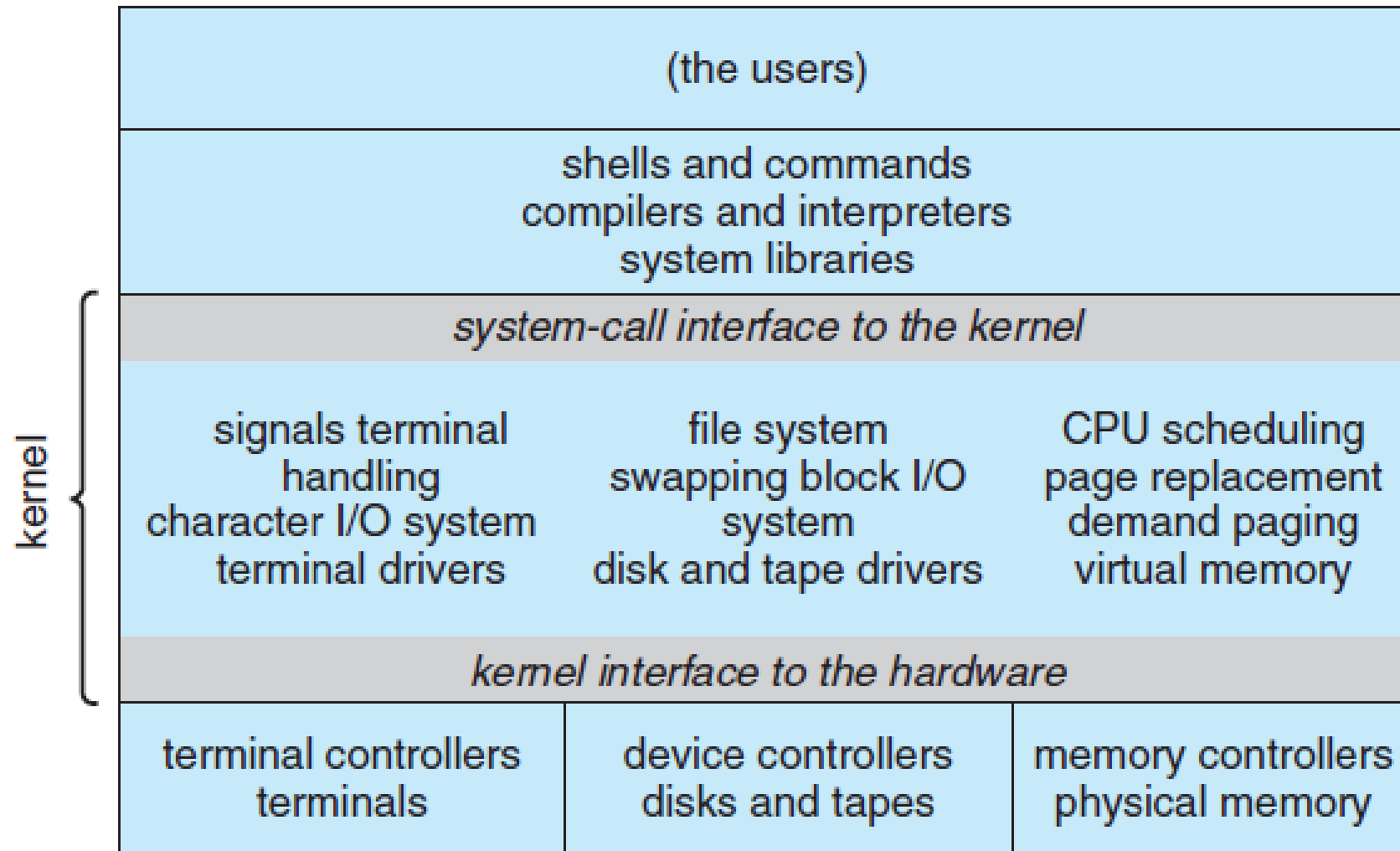


Figure 2.12 Traditional UNIX system structure.



Layered Approach

- To overcome the limitations of monolithic systems (like UNIX), designers break the OS into smaller, more manageable pieces.
- This provides greater control and allows programmers to change inner workings easily.

How it Works (The Levels):

The OS is organized into a number of discrete layers (levels).

- **Layer 0 (Bottom):** The physical hardware.
- **Layer N (Top):** The user interface.

The Rule of Interaction:

- A typical layer (Layer M) consists of data structures and routines.
- **Crucial Rule:** Layer M can *only* invoke operations on lower-level layers, and can only be invoked by higher-level layers.

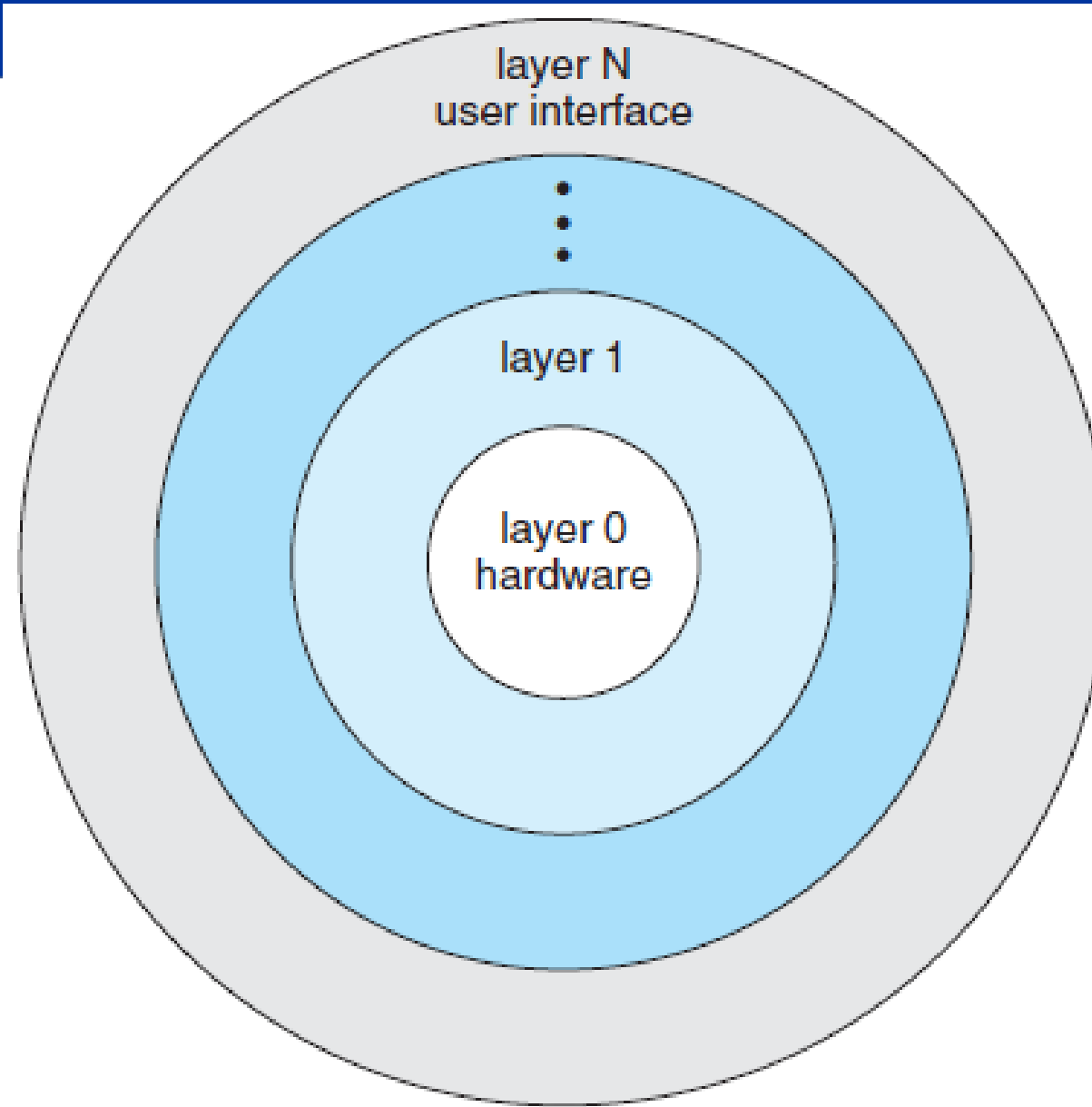


Figure 2.13 A layered operating system.



Advantages of Layering

1. **Simplicity of Construction:**

- Because a layer only uses the functions of the layers below it, the design is highly organized.

2. **Easier Debugging (Bottom-Up):**

- Developers can debug the first layer (Layer 1) using just the hardware (Layer 0).
- Once Layer 1 is proven correct, they move to Layer 2. If Layer 2 crashes, they know the bug is in Layer 2, because Layer 1 was already verified.

3. **Information Hiding:**

- Each layer hides the complex details of its data structures and hardware from the higher-level layers.
- A layer only needs to know *what* the lower operations do, not *how* they are implemented.

Challenges & Drawbacks of Layering

Challenge 1: Defining the Layers Appropriately:

- Because a layer can *only* use lower layers, careful planning is required.
- *The Dilemma:* The CPU scheduler needs to be below the backing-store driver (so the driver can wait for I/O). But if the CPU scheduler needs to swap data in and out of memory, it must be *above* the backing-store driver. It becomes a circular dependency!

Challenge 2: Performance Overhead (Efficiency):

- Layered implementations tend to be less efficient.
- When a user program executes an I/O operation, the system call must pass through the I/O layer, memory layer, CPU layer, etc., before hitting hardware. Each layer adds processing time.

Modern Trend: Due to these limitations, modern systems use fewer layers with more functionality to avoid the performance hit.

Microkernels

The Problem: As operating systems (like UNIX) expanded, their monolithic kernels became massive and difficult to manage.

The Microkernel Solution (e.g., Mach):

Structures the OS by removing all **nonessential components** from the kernel.

Implements these removed components as system and user-level programs in **User Space**.

What stays in the Kernel?

Only the absolute basics: minimal process and memory management, and a communication facility.



How it Works (Message Passing):

- Client programs and services **never interact directly**.
- Instead, they communicate indirectly by exchanging messages through the microkernel.
- *Example:* If a program wants to read a file, it sends a message to the microkernel, which passes it to the file server running in user space.

Microkernels

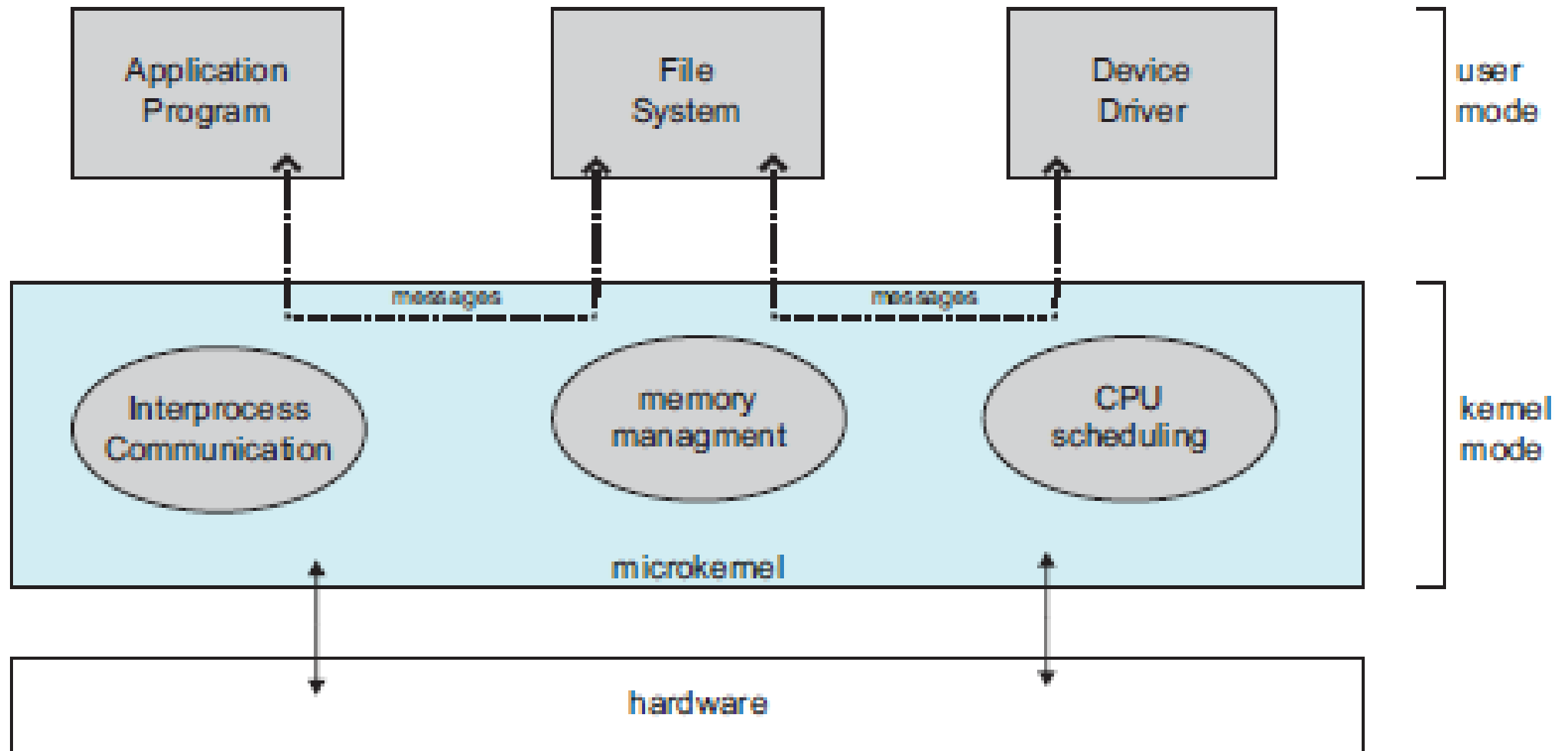


Figure 2.14 Architecture of a typical microkernel.

The Benefits:

- **Extensibility:** Very easy to extend the OS. New services are added to user space without modifying the core kernel.
- **Portability:** A smaller kernel is much easier to port to new hardware architectures.
- **Security & Reliability:** Most services run as user processes. If a service (like a device driver) fails, it crashes independently; the rest of the OS remains untouched.

The Drawback (Performance):

- System performance can suffer due to **increased overhead**. Constantly passing messages back and forth between user space and kernel space takes time.



Real-World Examples:

- **Mach:** Developed at Carnegie Mellon University in the mid-1980s.
- **Mac OS X:** The core kernel (Darwin) is partly based on the Mach microkernel.
- **QNX:** A real-time OS for embedded systems.
- **Windows NT (A Cautionary Tale):** Started with a layered microkernel, but performance was poor compared to Windows 95. Microsoft eventually moved layers back into the kernel (in Windows NT 4.0 and XP) to regain speed.



Loadable Kernel Modules

- Using **loadable kernel modules (LKMs)** is widely considered the best current methodology for operating-system design.
- Used by modern UNIX (Solaris, Linux, Mac OS X) and Windows.

The Design Philosophy:

- **Core Services:** The kernel only provides a set of essential core components (e.g., CPU scheduling and memory management).
- **Dynamic Linking:** Additional services are linked in via modules, either at boot time or dynamically while the kernel is running.

The Primary Benefit:

- You do not need to recompile the entire kernel every time you want to add a new feature (like supporting a new file system or plugging in a new device).



Why Modules are Superior

1. Compared to the Layered Approach:

- *Similarity:* Like a layered system, each kernel section has defined, protected interfaces.
- *Advantage: **More Flexible.*** Unlike strict layering, any module can call any other module directly without passing through intermediate levels.

2. Compared to the Microkernel Approach:

- *Similarity:* The primary core module only has basic functions and knowledge of how to load and communicate with other modules.
- *Advantage: **More Efficient.*** Modules communicate directly; they do not have to invoke slow *message passing* back and forth between user space and kernel space.

Real-World Example (Solaris)

The Solaris Operating System:

Organized around a core kernel with seven specific types of loadable modules:

1. Scheduling classes
2. File systems
3. Loadable system calls
4. Executable formats
5. STREAMS modules
6. Miscellaneous
7. Device and bus drivers

Linux Implementation:

Linux also heavily uses loadable kernel modules, primarily to support various device drivers and file systems dynamically.

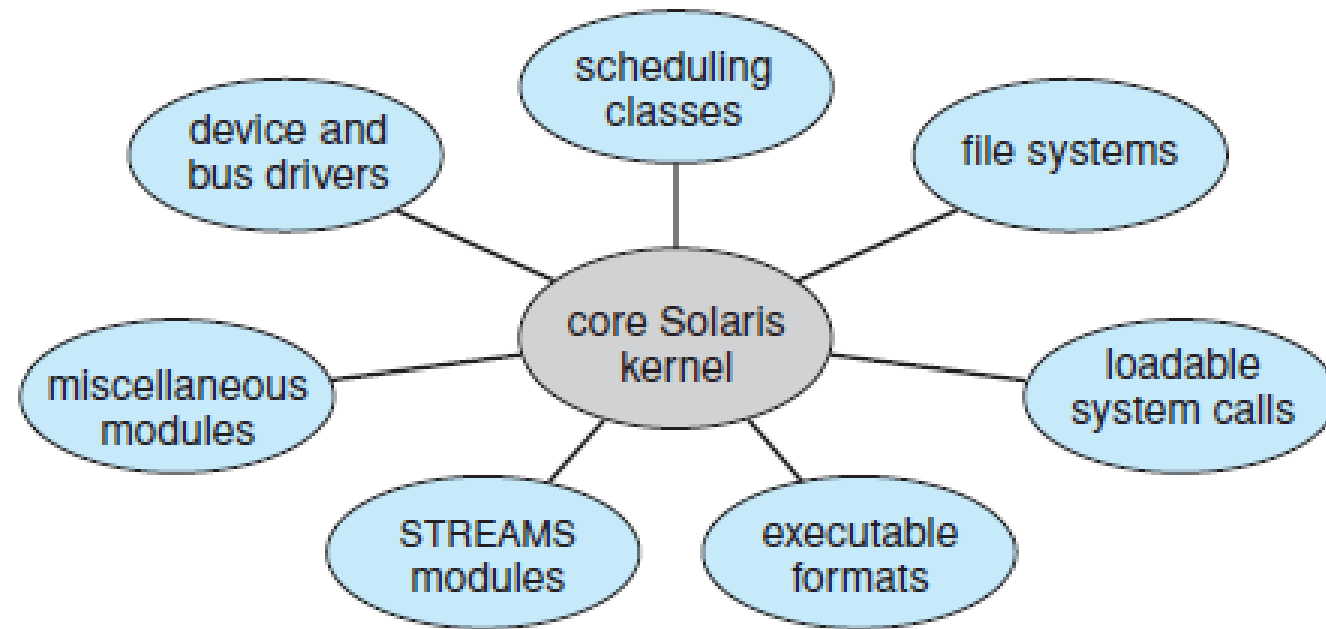


Figure 2.15 Solaris loadable modules.



- In practice, very few operating systems adopt a single, strictly defined structure (like pure monolithic or pure microkernel).
- Instead, they combine different structures to form **Hybrid Systems**.

The Goal:

- To balance and optimize **performance, security, and usability**.

Examples of Hybrids:

- **Linux & Solaris:** Mostly **Monolithic** (keeping the OS in a single address space for blazing-fast performance) but also **Modular** (so new functionality can be dynamically added).
- **Windows:** Largely **Monolithic** (for performance) but retains **Microkernel** behaviors (to support separate user-mode subsystems) and uses **loadable modules**

Apple's Mac OS X uses a highly successful layered, hybrid structure.

The Top Layers (User Environment):

- **Aqua UI:** The signature, user-friendly graphical user interface.
- **Application Environments:** The layer where apps run.
- **Cocoa:** The primary environment that specifies an API for the Objective-C programming language (used to write native Mac applications).

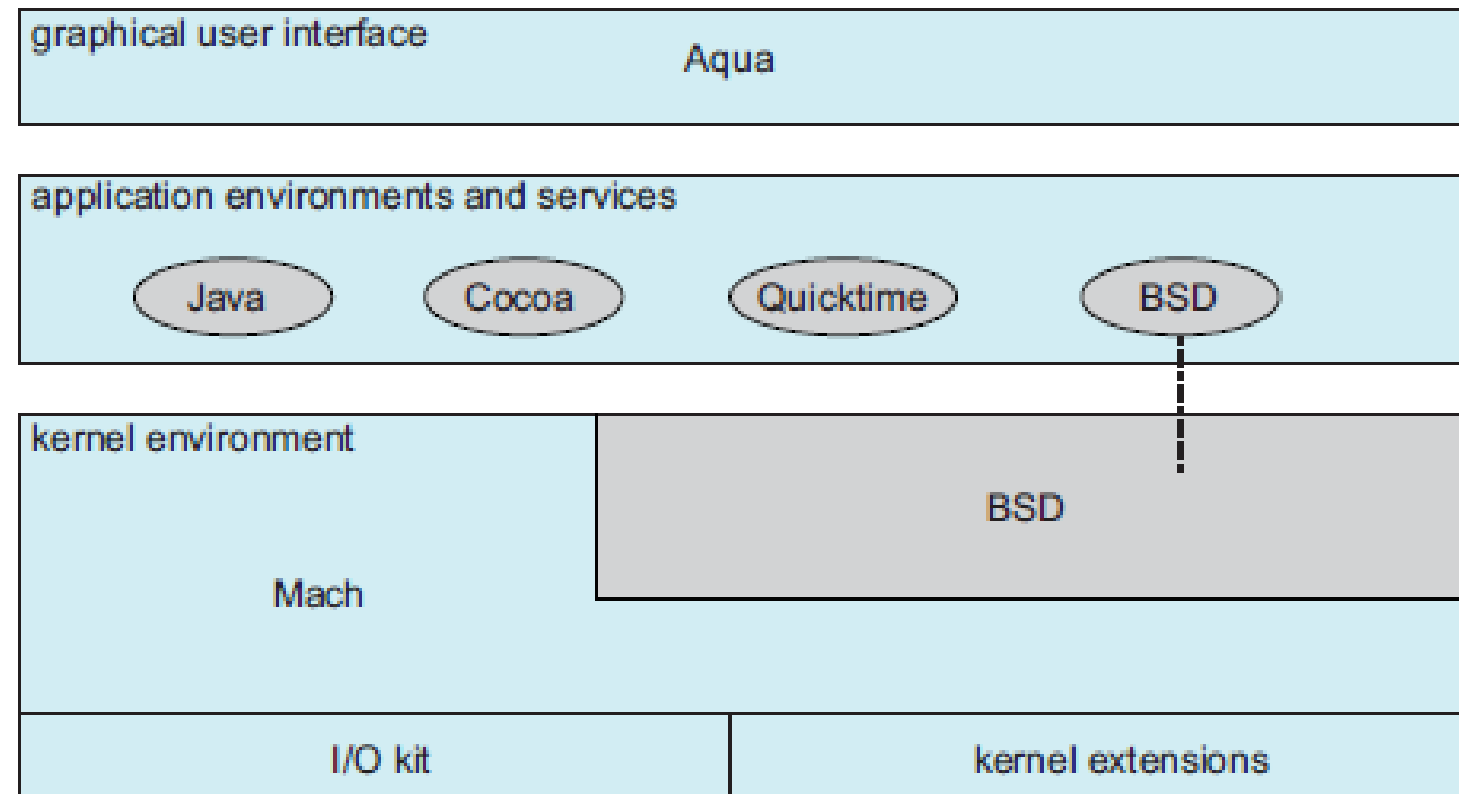


Figure 2.16 The Mac OS X structure.



Hybrid Systems- Mac OS X

Beneath the GUI, Mac OS X relies on a hybrid kernel combining two distinct systems:

- 1. Mach Microkernel:** Handles core hardware-level tasks: memory management, thread scheduling, and Interprocess Communication (IPC / message passing).
- 2. BSD UNIX Kernel:** Handles standard OS features: the command-line interface, networking, file systems, and POSIX APIs (like Pthreads).

Modularity & Extensions:

- **I/O Kit:** Provided for the development of device drivers.
- **Kernel Extensions:** Mac OS X's term for dynamically loadable kernel modules.



Closed-source and structured on the Mac OS X operating system, but modified for mobile.

The 4-Layer Architecture (Top to Bottom):

- 1. Cocoa Touch:** The top-level API for Objective-C. It provides frameworks for apps and crucially adds support for mobile hardware features like **touchscreens**.
- 2. Media Services:** Provides services for graphics, audio, and video.
- 3. Core Services:** Provides underlying features like support for cloud computing and databases.
- 4. Core OS:** The foundation, based on the Mac OS X kernel environment (Mach microkernel + BSD UNIX).

Hybrid Systems- Apple iOS

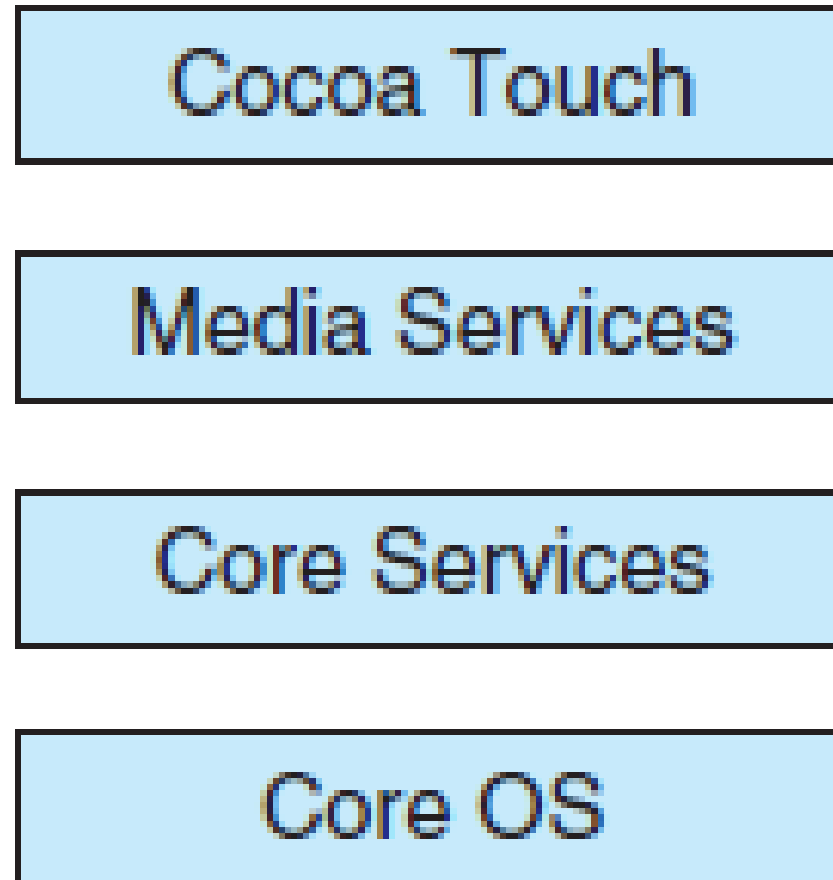


Figure 2.17 Architecture of Apple's iOS.



- Designed by the Open Handset Alliance (led by Google).
- **Open-source**, which fueled its rapid rise in popularity across various mobile hardware platforms.

The Layered Software Stack (Bottom to Top):

1. **Linux Kernel (Modified):** Handles process, memory, and device drivers, and is expanded to include heavy **power management** for battery life.
2. **Libraries:** Includes webkit (web browsing), SQLite (database), and multimedia.
3. **Android Runtime (Dalvik VM):**
 - Apps are written in Java using a custom Android API.
 - Java code is compiled into an executable file that runs on the **Dalvik virtual machine**, which is highly optimized for devices with limited memory and CPU power.
4. **Application Framework & Apps:** The top layers providing the rich frameworks developers use to build user applications.

Hybrid Systems- Google Android

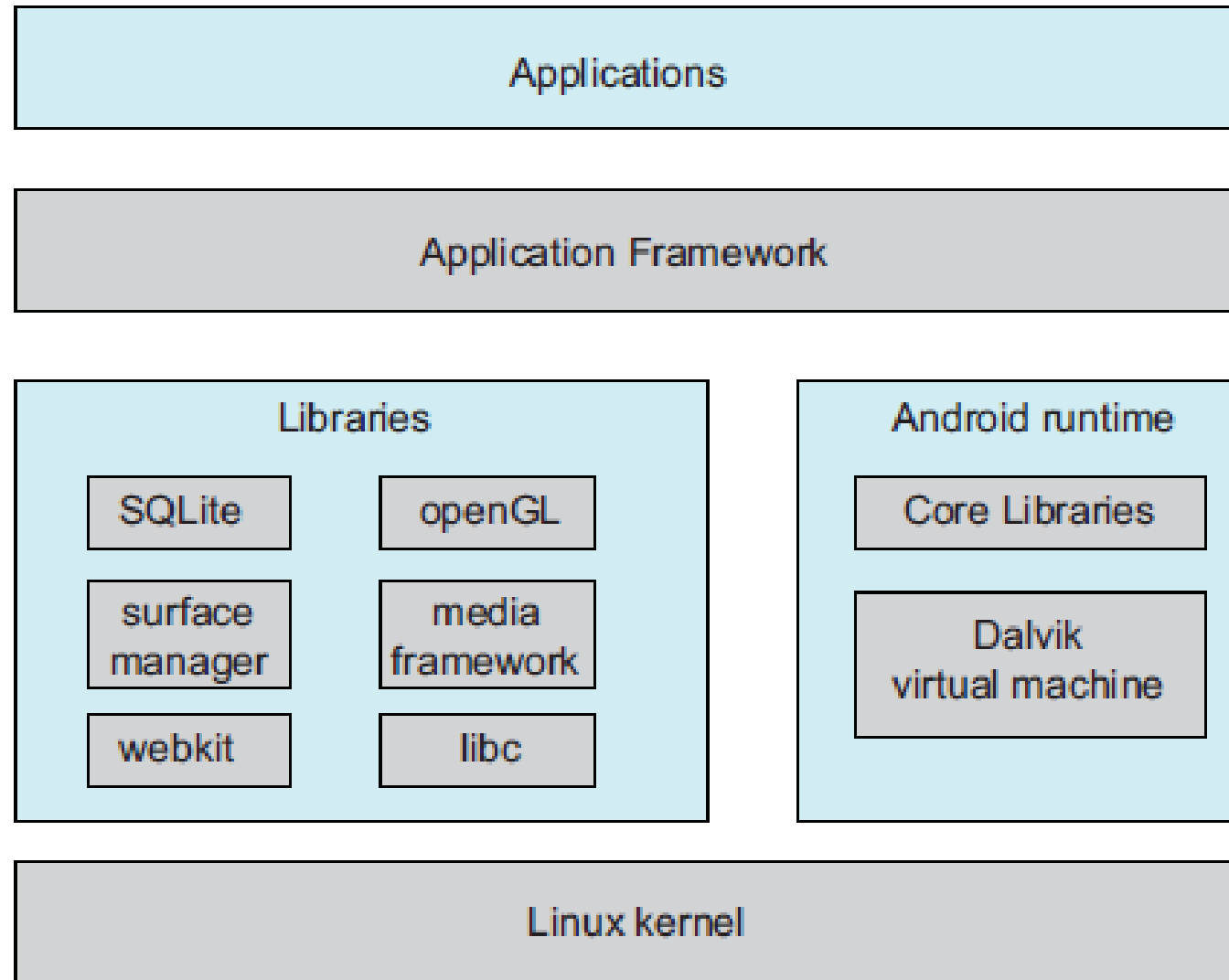


Figure 2.18 Architecture of Google's Android.



What is Booting?

- The procedure of starting a computer by locating the Operating System kernel, loading it into main memory, and starting its execution.

The Trigger (The Reset Event):

- When a computer is powered up or rebooted, the CPU instruction register is loaded with a predefined memory location, and execution starts there.

The Bootstrap Program:

- A small piece of code located at that predefined location.
- It is stored in **ROM** because RAM is in an unknown state at startup.

Initial Tasks of the Bootstrap:

- **Diagnostics:** Checks state of the machine to ensure hardware is working.
- **Initialization:** Sets CPU registers, device controllers, and main memory.
- **Locate OS:** Finds and starts the operating system.



Where Does the OS Live?

Small Systems (OS in ROM):

- **Examples:** Cellular phones, tablets, and game consoles.
- **How it works:** The entire OS is stored directly in ROM (also known as **firmware**).
- *Pros:* Rugged and simple.
- *Cons:* Changing the OS requires changing the hardware chips (or flashing an EPROM). Firmware is also slower and more expensive than RAM, so the OS is often copied to RAM for faster execution.

Large Systems (OS on Disk):

- **Examples:** Windows, Mac OS X, UNIX.
- **How it works:** The initial bootstrap loader is in firmware, but the actual Operating System is stored on a hard disk.
- *Pros:* The OS can be easily updated or changed by simply writing new versions to the disk.



The Two-Step Disk Boot Process

On large systems, the bootstrap code in ROM is too small to know how to load a massive, complex OS from a hard drive.

The Solution (The Two-Step Process):

- **Step 1 (The Boot Block):** Firmware bootstrap reads single block (block zero) from disk into memory. This is called **boot block**.
- **Step 2 (The Full Bootstrap):** The boot block contains simple code that knows the exact address and length of the *full* bootstrap program on the disk (e.g., **GRUB** for Linux).

System Running:

- Once the full bootstrap program is loaded, it traverses the file system, finds the OS kernel, loads it into memory, and starts execution.
- A disk containing boot partition is called **boot disk** or **system disk**.